08 線性時間排序 解答

08-E01 比較排序下界

推導

二元決策樹高度 h 最多有 2^h 個葉。要區分 24 種可能排列，需 2^h>=24。2^4=16<24、2^5=32>=24，因此 h>=5。

結論與檢核

比較模型下最壞至少 5 次。counting sort 不是僅靠比較取得次序，它讀取整數鍵使用計數陣列，所以不受這個模型下界限制。

08-E02 穩定 counting 放置

推導

count[0..2]=[1,1,3]，累積後 output 的結尾位置為 [1,2,5]。由右向左處理：(2,e) 放位置 5、pos[2]=4；(1,d) 放位置 2、pos[1]=1；(2,c) 放位置 4；(0,b) 放位置 1；(2,a) 放位置 3。

結論與檢核

輸出 [(0,b),(1,d),(2,a),(2,c),(2,e)]。同鍵 a、c、e 保持輸入次序，因為較後記錄先取得較右位置，較前記錄取得較左位置。

08-E03 radix 成本取捨

推導

8-bit digits 的成本代理為 4(n+256)=4n+1024；16-bit digits 為 2(n+65536)=2n+131072。若單位成本相同，後者較小等價於 2n>130048，即 n>65024。A 初始化固定項較小，B 每元素掃描次數較少；不能把小 n 與大 n 的方向說反。這只是模型比較，不是實測門檻。

結論與檢核

不能只用 d 少就判 B 勝，因為每輪 counting 表初始化成本 r 納入公式。實際還需考慮記憶體、快取、鍵分布、穩定分類成本與語言實作。

08-E04 穩定性失效的精確位置

推導

初始前綴C=[1,2,5]。正向依序把2a放5、0b放1、2c放4、1d放2、2e放3，得到[0b,1d,2e,2c,2a]，鍵雖有序，同鍵identity反轉。[12,11]個位排序得[11,12]，十位都1；若第二輪反轉tie，就變[12,11]而錯。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

08 線性時間排序 解答 （續）

結論與檢核

反向回填是末位置後減一規格下的穩定方法；正向掃描若改用起始位置後加一也能穩定。LSD歸納步需要同一當前位群保留先前低位排序，正是穩定性所提供。

08-E05 非均勻桶機率與平方成本

推導

展開同桶有序記錄對：$\Sigma N_i^2 = n + \Sigma_{(j \neq k)} I[j,k\text{同桶}]$，獨立時同桶機率 $\Sigma p_i^2 = 1/4 + 1/16 + 1/16 = 3/8$。故期望 $4 + 4 \times 3 \times 3/8 = 17/2$；均勻四桶為 $4 + 12/4 = 7$。

結論與檢核

均勻抽$J \in 0..3$，令所有4筆落J，每桶期望大小仍1，但每次$\Sigma N_i^2 = 16$。不能以$(E[N_i])^2$取代$E[N_i^2]$，也不能用邊際均勻取代記錄間獨立。這是平方工作上界的反例，並不表示相等值本身會使穩定insertion sort達平方時間。

08-E06 前綴計數查詢與負鍵平移

推導

鍵v映射v+2，頻率依−2,−1,0,1,2,3為[2,0,1,2,0,1]，保存前綴P=[2,2,3,5,5,6]。區間[−1,1]是P[3]−P[0]=5−2=3；[−2,−2]是P[0]−P[−1]=2−0=2，其中P[−1]=0是邊界慣例。

結論與檢核

回填每放一筆就把C[key]減一，已不再表示≤該鍵的原始總筆數；先複製P再回填。預處理Θ(n+範圍大小)，每次查詢O(1)，平移不會壓縮巨大稀疏鍵範圍。

08 線性時間排序 解答 （續）

原創驗算程式附錄: counting_sort.py

```
"""Stable counting sort for nonnegative integer keys."""

def counting_sort(values, key_max):
    counts = [0] * (key_max + 1)
    for value in values:
        counts[value] += 1
    for i in range(1, len(counts)):
        counts[i] += counts[i - 1]
    result = [0] * len(values)
    for value in reversed(values):
        result[counts[value] - 1] = value
        counts[value] -= 1
    return result


sample = [2, 0, 2, 1, 2]
assert counting_sort(sample, 2) == [0, 1, 2, 2, 2]
print(counting_sort(sample, 2))
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

08 線性時間排序 解答 (續)

原創驗算程式附錄: linear_sort_checks.py

```
"""Stable identity traces and exact finite bucket-distribution calculations."""
from itertools import product
from fractions import Fraction


def counting(records, key_max, key=lambda record: record[0], reverse=True):
    counts=[0]*(key_max+1)
    for record in records:
        counts[key(record)]+=1
    frequencies=counts[:]
    for i in range(1,len(counts)):
        counts[i]+=counts[i-1]
    prefix=counts[:]
    output=[None]*len(records)
    placements=[]
    indices=range(len(records)-1,-1,-1) if reverse else range(len(records))
    for j in indices:
        value=key(records[j])
        slot=counts[value]-1
        output[slot]=records[j]
        counts[value]-=1
        placements.append((j+1,records[j],slot+1,counts[:]))
    return output,frequencies,prefix,placements


def radix(records,base,digits):
    values=list(records)
    states=[values[:]]
    power=1
    for _ in range(digits):
        values=counting(values,base-1,key=lambda r:(r[0]//power)%base)[0]
        states.append(values[:])
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

08 線性時間排序 解答 （續）

```
        power*=base
    return states


def insertion(values):
    result=list(values)
    moves=0
    for j in range(1,len(result)):
        value=result[j]
        i=j-1
        while i>=0 and result[i][0]>value[0]:
            result[i+1]=result[i]
            i-=1
            moves+=1
        result[i+1]=value
    return result,moves


def bucket(records):
    n=len(records)
    if not n:
        return [],[],0
    buckets=[[] for _ in range(n)]
    for record in records:
        assert 0<=record[0]<1
        index=(n*record[0]).numerator//(n*record[0]).denominator
        buckets[index].append(record)
    sorted_buckets=[]
    moves=0
    for group in buckets:
        ordered,cost=insertion(group)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

08 線性時間排序 解答 (續)

```
        sorted_buckets.append(ordered)
        moves+=cost
    return [r for group in sorted_buckets for r in group],sorted_buckets,moves


def main():
    checked=0
    for n in range(8):
        for keys in product(range(3),repeat=n):
            records=list(zip(keys,range(n)))
            ordered=counting(records,2)[0]
            assert ordered==sorted(records,key=lambda r:r[0])
            checked+=1
    sample=[(2,'a'),(0,'b'),(2,'c'),(1,'d'),(2,'e')]
    ordered,freq,prefix,placements=counting(sample,2)
    assert freq==[1,1,3] and prefix==[1,2,5]
    assert [p[2] for p in placements]==[5,2,4,1,3]
    assert counting(sample,2,reverse=False)[0]==[(0,'b'),(1,'d'),(2,'e'),(2,'c'),(2,'a')]
    values=[412,205,421,115,212,405]
    states=radix(list(zip(values,range(6))),10,3)
    assert [[r[0] for r in s] for s in states]==[values,[421,412,212,205,115,405],
[205,405,412,212,115,421],[115,205,212,405,412,421]]
    radix_cases=0
    for n in range(5):
        for keys in product([0,1,3,4,11,15],repeat=n):
            records=list(zip(keys,range(n)))
            for base,digits in [(2,4),(4,2),(10,2)]:
                assert radix(records,base,digits)[-1]==sorted(records,key=lambda r:r[0])
            radix_cases+=1
    assignments=0
    for n in range(1,6):
        square_total=0
        for cells in product(range(n),repeat=n):
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

08 線性時間排序 解答 （續）

```
            sizes=[cells.count(i) for i in range(n)]
            square_total+=sum(size*size for size in sizes)
            assignments+=1
        assert Fraction(square_total,n**n)==2*n-1
    probabilities=[Fraction(1,2),Fraction(1,4),Fraction(1,4),Fraction(0)]
    expected=Fraction(0)
    for cells in product(range(4),repeat=4):
        probability=Fraction(1)
        for cell in cells:
            probability*=probabilities[cell]
        expected+=probability*sum(cells.count(i)**2 for i in range(4))
    assert expected==Fraction(17,2)
    for n in range(2,40):
        descending=[(Fraction(n-j,2*n*n),j) for j in range(n)]
        ordered,groups,moves=bucket(descending)
        assert ordered==sorted(descending,key=lambda r:r[0])
        assert len(groups[0])==n and moves==n*(n-1)//2
    bucket_cases=0
    for n in range(7):
        for keys in
product([Fraction(0),Fraction(1,5),Fraction(1,2),Fraction(9,10)],repeat=n):
            records=list(zip(keys,range(n)))
            assert bucket(records)[0]==sorted(records,key=lambda r:r[0])
            bucket_cases+=1
    assert 4*65024+1024==2*65024+131072
    print(f"PASS stable_count_arrays={checked} radix_arrays={radix_cases}
exact_uniform_bucket_assignments={assignments} nonuniform_square_moment={expected}
bucket_arrays={bucket_cases} worst_descending_bucket=True")


if __name__=="__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study